

Kyambogo University
Department of Biomedical and Mechatronics
Engineering
TEMB 1202: Computing II Assignment
Project Report:
Prepaid Electricity Token System

Group 4

27th May, 2026

Table 1: Group 4

Name	Registration Number
BUKENYA JOHN MATHIAS	25/U/BIO/025/GV
SSEKYANZI EDRINE ALLAN	25/U/BIO/030/GV
NAMBATYA BUSHIRAT	25/U/BIO/01405/PD
MWESIGWA ELLY	25/U/BIO/01402/PD

Contents

1	Abstract	3
2	Introduction	3
3	Procedures and Observations	4
4	Details and Logic	
	The Problem Being Solved	5
5	System Features	5
6	Program Design and Structure	5
	6.1 User-Defined Functions	6
7	Design Decisions and Justifications	6
8	Summary of Results	6
	8.1 Calculations	7
9	Challenges	7
10	Discussion	8
11	Conclusion	9

1 Abstract

This report describes the development of a C program designed to manage electricity utility records and token purchases. The system utilizes structured data and modular programming to handle customer registration, security authentication and automated billing across three consumption categories. Numerical verification of the billing algorithm showed accurate unit allocation after service fee deductions.

2 Introduction

The goal of this project was to create a software interface for electricity distribution management. Modern utility systems require high accuracy in billing and data persistence. The project focused on implementing core C concepts like such as variables, conditional statements, loops, functions, arrays, pointers, structures covering user-defined functions, data structures (struct) and constant definitions, to solve real-world engineering data management problems.

3 Procedures and Observations

The system was built in four distinct phases.

First, security was established using the `string.h` library to verify user credentials using a simple login function.

Second, a data management layer was created using a struct to group diverse customer data types into a single array database. Since we are managing multiple pieces of related data for each person, we used a C Structure (struct) to group these variables. Then, created an Array of these structures to hold your entire customer list .

Third, a menu-driven interface was implemented using a `while(1)` loop to keep the program running continuously and switch cases.

Finally, a calculation engine was programmed to apply specific rates based on the customer's category (Domestic, Commercial, or Industrial). It was observed that utilizing `#define` constants significantly reduced syntax errors when updating utility rates. We also used conditional logic (if/else) to check the customer category and apply the correct electricity rate with logical operators as well.

4 Details and Logic

The Problem Being Solved

Managing electricity prepaid tokens manually or without a structured system is prone to errors, data loss and difficulty in tracking individual usage. The goal of this project was to develop a digital, menu-driven C program to automate customer registration, secure token purchasing and monitor electricity usage levels.

5 System Features

- **Secure Authentication:** Protects the system with a simple login mechanism.
- **Customer Management:** CRUD operations (Create, Read, Update, Delete) to manage customer profiles.
- **Token Purchase Logic:** Calculates units purchased based on customer categories (Domestic, Commercial, Industrial) after deducting a fixed service fee.
- **Automated Warnings:** Provides alerts for low unit balances and energy-saving tips for high usage.

6 Program Design and Structure

The program is built using a procedural programming approach in C, emphasizing modularity.

- **Data Structures:** Uses a `struct` to group related customer information (name, meter_number, phone_number, category, remaining_units)
- **Storage:** Employs an array of structures to manage up to 100 customers.
- **Control Flow:** Utilizes a `while(1)` main loop with a `switch` statement for the menu-driven interface.

6.1 User-Defined Functions

- `login()`: Handles initial authentication.
- `findCustomerByMeter()`: A utility function to search the array for specific records.
- CRUD functions: `registerCustomer()`, `searchCustomer()`, `updateCustomerDetails()`, `deleteCustomerRecords()`.
- `buyTokens()`: Handles financial calculations, rate application, and token generation.
- `displayCustomer()`: Handles formatted output.

7 Design Decisions and Justifications

- **C Structs:** Used to organize customer data into a single entity, improving readability and maintenance.
- **Constants (#define):** Defined rates and thresholds centrally to make future updates simple.
- **Modularization:** Breaking logic into small, single-purpose functions simplified debugging.

8 Summary of Results

The following configuration parameters were successfully implemented:

- Domestic Rate: 700 UGX/kWh
- Commercial Rate: 900 UGX/kWh
- Industrial Rate: 600 UGX/kWh
- Fixed Service Fee: 1000 UGX

8.1 Calculations

The system calculates units based on the formula:

$$UnitsPurchased = \frac{AmountPaid - ServiceFee}{CategoryRate} \quad (1)$$

Sample calculation for a Commercial customer paying 50,000 UGX:

$$Units = \frac{50,000 - 1,000}{900} = \frac{49,000}{900} \approx 54.44kWh \quad (2)$$

9 Challenges

Developing this project involved overcoming several technical challenges, starting with the mastery of `<string.h>` for comparisons and `<stdlib.h>` for random number generation, alongside data validation to prevent potential database corruption. Managing multiple customers through complex structures proved difficult, as did ensuring precision when calculating units based on categories and applying specific rates. The implementation required careful logic flow management for critical operations—including updating, deleting and searching records, as well as accurately tracking remaining unit balances after each purchase while simultaneously handling robust input validation to ensure all user entries remained correct. Finally, we successfully integrated this logic with the LaTeX interface, learning to effectively structure the code for the markup language.

10 Discussion

The implementation of `<string.h>` and `<stdlib.h>` provided the necessary tools for authentication and random token generation. we learnt that The `strcmp` function is a built-in tool in the C Standard Library (found in the `<string.h>` header) used to compare two strings. It compares them character by character based on their ASCII values until it finds a difference or reaches the end of a string.

In C programming, the `rand()` function is used to generate pseudo-random integers ranging from 0 to a system-defined maximum called `RAND_MAX`. It is part of the standard library and requires including `<stdlib.h>`. The use of user-defined functions like `buyTokens()` and `findCustomerByMeter()` allowed for code reusability and modular testing. A primary difficulty encountered was the volatility of the array data; since file I/O was not implemented in this phase, all records are lost upon termination. Future improvements should include a `.txt` or `.csv` database integration for persistence.

11 Conclusion

The Prepaid Electricity Token System successfully addresses the need for a structured customer and token management tool. It fulfills all functional requirements, provides a clean user interface and demonstrates the practical application of C programming concepts including structures, arrays, and modular function design.